

Testabilidade: dependências, acoplamento e design

Live de Python # 306

Sem flame!



Essa live pode levar a discussões teóricas por diferentes correntes de pensamentos em programação. Principalmente em relação a técnicas.

Quero que pensemos em python como uma linguagem dinâmica em runtime, com suporte a tipagem gradual, multi-paradigma e também em uma corrente de pensamento pós lineu.

Vamos tentar espremer a linguagem, sem pensar em técnicas de correntes específicas.

O que quer dizer sem dogmatismo de OOP/Funcional/Tipagem estática/... Sem agradar livros de engenharia.

Outro aviso



Vamos usar Pytest como parte elementar dessa discução. Não pretendo me aprofundar nele. Alguns materiais sobre ele:

- Pytest: Uma introdução - Live de Python #167
- Pytest Fixtures - Live de Python #168

Se quiser um básico sobre testes:

- O mínimo que você deveria saber sobre testes unitários
- Uma introdução aos testes: Como fazer?| Live de Python #232

Então... antes de tudo:

```
pip install pytest
```

Roteiro



1. Um passo pra trás

Entendendo as problemáticas

2. A solução de design é **depende**

Design, DI, MP, IoC

3. Variações de problemas

Constantes e logs

4. A pureza dos puros

IoC e seus problemas

Entendendo o problema

Um passo para trás

Design de código



Uma definição:

> O design é definido tanto como "o processo de definir a arquitetura, os componentes, as interfaces e outras características de um sistema ou componente" quanto como "o resultado desse processo".

BOURQUE, Pierre; FAIRLEY, R. E. (ORGS.). SWEBOK: guide to the software engineering body of knowledge. Version 3.0 ed. Los Alamitos, CA: IEEE Computer Society, 2014.

Outra que eu gosto:

> O processo de design de sistemas aloca os requisitos a sistemas de hardware ou software. Ele estabelece uma arquitetura geral do sistema. O projeto de software envolve a identificação e a descrição das abstrações fundamentais do sistema de software e seus relacionamentos.

SOMMERVILLE, Ian. Software engineering. Tenth edition ed. Pearson, 2016.

Definições



Uma outra boa:

> Testar antes (TDD) também é uma técnica de design. (...) Isso é design lógico, ou de interface — exatamente como meus antigos livros da faculdade explicavam. Claro, esses livros nunca explicavam como fazer design lógico; eles apenas deixavam claro que ele precisava ser feito, e que precisava ser feito antes do design físico e da implementação, e que Deus ajude quem confundisse as duas coisas.

BECK, K. Aim, fire. IEEE Software, v. 18, n. 5, p. 87–89, set. 2001.

Um cenário lúdico



Imagine que temos que ligar o cooler do nosso PC quando temperatura do processador for maior que `TEMPERATURA_LIMITE` e registrar essas alterações junto da temperatura atual.

Temos que:

1. Saber a temperatura atual do processador
2. Checar se a temperatura atual exige que cooler seja ligado
3. Logar se o cooler será ligado ou desligado e qual a temperatura atual
4. Retornar se deve ligar ou desligar

Pensando em código



O que nos daria algo como isso (ex_00.py):

```
def ligar_cooler() -> bool:
    temp_atual = psutil.sensors_temperatures()['coretemp'][0].current #???

    if temp_atual >= TEMPERATURA_LIMITE:                    # ???
        logger.info(f'Cooler ligado | TEMP {temp_atual}')   # ???
        return True

    logger.info(f'Cooler desligado |  TEMP {temp_atual}')   # ???
    return False
```

Aqui vemos um caso clássico de um **mar de dependências** que ao primeiro momento parecem inofensivas.

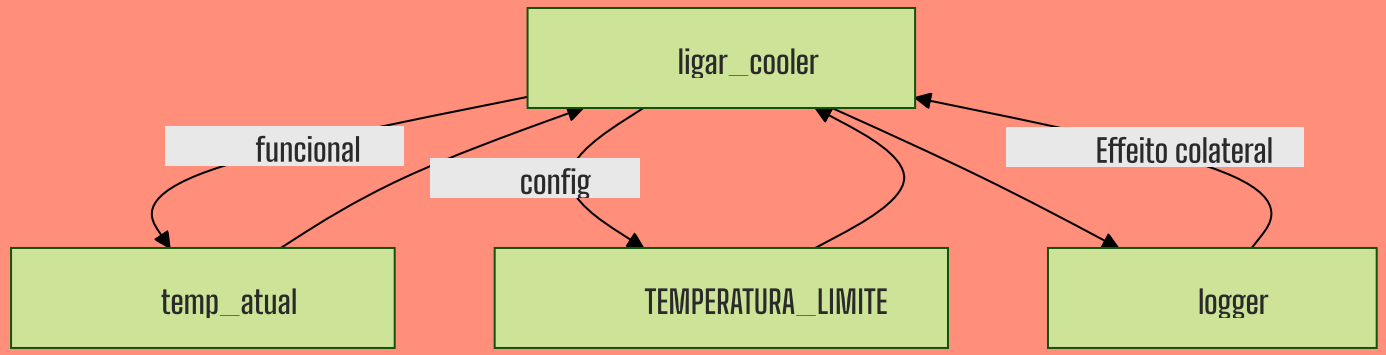
```
pip install psutil
```

Olhando com atenção

```python
def ligar_cooler() -> bool:
    temp_atual = psutil.sensors_temperatures()['coretemp'][0].current #???

    if temp_atual >= TEMPERATURA_LIMITE:                    # ???
        logger.info(f'Cooler ligado | TEMP {temp_atual}')   # ???
        return True

    logger.info(f'Cooler desligado |  TEMP {temp_atual}')   # ???
    return False
```



Dando nomes



Definições informais...

Dependência

A dependência é uma relação entre partes do código onde uma não pode funcionar sem a outra. Em um cenário de dependência, um módulo ou função depende diretamente de outro para executar seu comportamento.

Acomplamento

O acoplamento mede o grau de dependência entre as partes do código. Quanto maior for o acoplamento, mais difícil será alterar ou manter o código, já que mudanças em uma parte podem afetar outras.

Dependência / Acoplamento

Aqui vemos um caso clássico de um *mar* de dependências que ao primeiro momento parecem inofensivas:

1. A função `ligar_cooler` depende de `psutil` para ser executada.
2. O corpo dela ainda depende do retorno de `psutil` para decidir em qual *branch* de código vai entrar.
3. `TEMPERATURA_LIMITE` é uma constante que não conhecemos dentro do escopo
4. `logger` não muda a lógica de resposta de `ligar_cooler`, mas pode gerar efeitos colaterais visíveis nos testes (stdout, arquivos de log), e testes podem precisar capturar ou ignorar isso.

Ou seja, para ser executada ela tem diversas dependências de coisas que não fazem parte do seu escopo e ela não tem nenhum conhecimento.

Os testes como design



`ligar_cooler` é idiomaticamente simples, correta, fácil de ler. E, claro..., simples de testar!

O que esperaríamos que fosse um teste *AAA* simples, como:

```
def test_ligar_cooler():
    expected_result = True              # Arrange
    result = ligar_cooler()             # Act - Chama o SUT
    assert expected_result == result    # Assert
```

O problema é que o teste passa a depender de estado externo e perde determinismo. Muitas coisas "interferem" no estado final do cooler.

Uma nota básica... - SUT/AAA



Partindo de uma premissa que temos que escrever testes, temos sempre algo a ser testado, um SUT (System Under Test).

```
def xpto():  # Função pura
    return 42
```

Para testarmos isso, existe um pequeno padrão. O AAA (Arrange-Act-Assert):

```
def test_xpto():
    expected_result = 24  # Arrange / Falha proposital

    result = xpto()  # Act

    assert result == expected_result  # Assert
```

Esse é o cenário ideal, perfeito, *by the book* dos testes.

Embora passe



`pytest -v <test>`: incrivelmente, "o teste passa" dessa forma. O fato de não conseguirmos cobrir 100% dele é somente um detalhe... Não dizem que 100% de cobertura é um mito?

```
def test_ligar_cooler():
    expected_result = True
    result = ligar_cooler()
    assert expected_result == result
```

Se baseia em diversas premissas falsas:

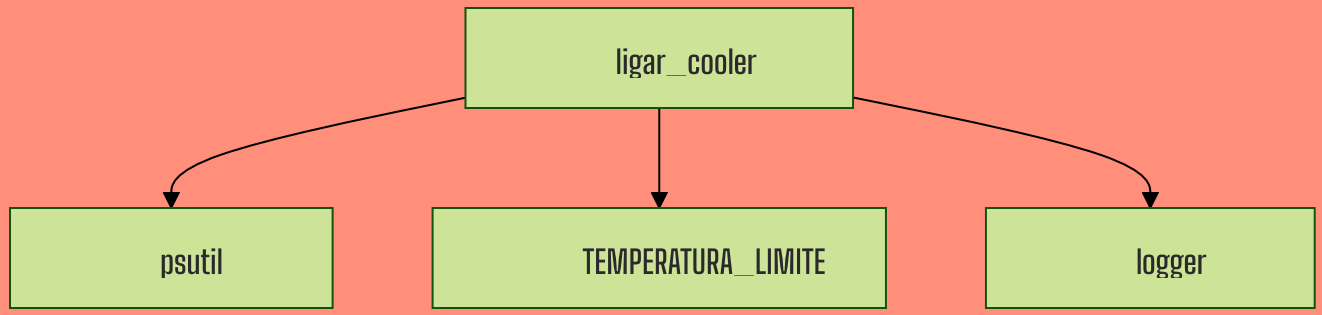
- `TEMPERATURA_LIMITE` sempre será `40`;
- `temp_atual` é sempre maior que `40`

Bom... Observações feitas, vamos começar a pensar um pouco sobre esse código e suas interações com coisas de fora da função.

Acoplamento



Todas essas coisas trabalham cordenadamente para produzir o comportamento correto. Mas temos muitas coisas acopladas:



- Acoplamento funcional: `ligar_cooler` precisa do valor retornado pelo `psutil`
- Acoplamento de configuração: a função depende de `TEMPERATURA_LIMITE`.
- Acoplamento de infra: `logger` vai gerar side effects malucos

O problema de alto acoplamento é que ele reduz a testabilidade e aumenta o risco de bugs quando o sistema cresce, porque qualquer mudança em uma dependência pode quebrar o SUT.

Design é

DEPENDE

A solução de design é "depende"



Aqui agora vão brotar teorias de todos os lados, sério... as pessoas têm MUITAS opiniões. E cada uma delas vai ter seus ônus e seus bônus...

- SoC / Desacoplar
- Injetar
- Patching
- Inverter o controle
  - Usar containers
- ...

SoC [ex_01.py]



Vamos começar com o problema básico do `psutil` e vamos nos aprofundando nos próximos depois.

```python
def cpu_temp() -> float:
    return psutil.sensors_temperatures()['coretemp'][0].current


def ligar_cooler() -> bool:
    temp_atual = cpu_temp()
    # ...
```

Vamos separar a preocupação (SoC) dos erros do `psutil`. Ali podem ocorrer uma soma de inúmeros problemas envolvendo SOs (alguns não têm essa implementação) e Sensores (às vezes não tem driver).

Mas não é nosso objetivo aqui, então... pulamos!

A solução mais simples, parâmetros [ex_02.py]



Quando pensamos um pouco além no design da aplicação.

Uma forma de conseguir testar isso, seria fazer com que **`ligar_cooler`** somente receba o valor de **`cpu_temp`**. Sem a necessidade explícita de acoplar o comportamento em seu corpo:

```
def ligar_cooler(temperatura_atual: float) -> bool:
    if temperatura_atual >= TEMPERATURA_LIMITE:
        logger.info(f'Cooler ligado | TEMP {temperatura_atual}')
        return True

    logger.info(f'Cooler desligado |  TEMP {temperatura_atual}')
    return False
```

A solução mais simples, parâmetros [ex_02.py]



O que poderia fazer o teste ser estupidamente simples:

```
@pytest.mark.parametrize(  # Arrange
    ('temp', 'expected_result'),
    [(30, False), (50, True)],
    ids=['Temperatura baixa', 'Temperatura alta']
)
def test_ligar_cooler(temp, expected_result):
    result = ligar_cooler(temp)       # Act
    assert expected_result == result  # Assert
```

Cobrindo ambos os casos. Tanto para ligar o cooler, quanto para não ligar! Isso é ótimo, não?

Agora a função tira de dentro dela a caixa preta que vinha com `cpu_temp`. A testabilidade aqui fica bastante atraente.

⚠ Shift



Embora pareça ótimo, isso tem uma pegadinha grande. Porque, de forma geral, só estamos movendo a "ruim de testar" para outro lugar.

No final das contas, para executar seu código, como ele não tem mais a dependência, ela vai aparecer de outra forma e em outro lugar:

```
if __name__ == '__main__':
    temp = cpu_temp()
    ligar_cooler(temp)
```

O que parecia bom, na verdade, é alguns chamam de "shift do problema". A dependência não foi removida, mas o local de resolução é alterado.

Mas, talvez exista uma forma diferente de resolver isso sem criar uma **dependência temporal**.

Injeção de dependência [ex_03.py]



Outra forma de resolver esse problema, baseado que temos HOFs (Funções de ordem superior) é usar a própria função como argumento.

O que continuaria fácil de testar e não levaria a complexidade para outro lugar do código:

```python
def ligar_cooler(cpu_temp_fn: Callable[[], float] = cpu_temp) -> bool:
    temperatura_atual = cpu_temp_fn()

    if temperatura_atual >= TEMPERATURA_LIMITE:
        logger.info(f'Cooler ligado | TEMP {temperatura_atual}')
        return True

    logger.info(f'Cooler desligado |  TEMP {temperatura_atual}')
    return False
```

Injeção de dependência [ex_03.py]



O que nos daria um teste parecido com esse:

```
@pytest.mark.parametrize(
    ('func', 'expected'),
    [(lambda: 30, False), (lambda: 50, True)],
    ids=['Temperatura baixa', 'Temperatura alta']
)
def test_ligar_cooler(func, expected):
    result = ligar_cooler(func)
    assert expected == result
```

Aqui temos uma qualidade interessante, não fizemos o shift para o main, a responsabilidade de execução continua em `ligar_cooler`, mas a função pode seguir de forma fluída.

⚠ ponto de atenção



Talvez um ponto de atenção que deve ser considerado nessa técnica é que a resolução dos argumentos default do python é feita no momento em que a função é definida, o que pode oferecer alguns problemas a depender dos testes que serão feitos.

Monkey-patch [ex_04.py]



Outra solução prática e muito comum, que gera menos "tensão" com o código original é fazer o *patch*, estamos numa linguagem dinâmica, isso não é ruim...

O pytest tem um fixture para isso chamada `monkeypatch`. Ficando assim:

```
@pytest.mark.parametrize(
    ('func', 'expected'),
    [(lambda: 30, False), (lambda: 50, True)],
    ids=['Temperatura baixa', 'Temperatura alta']
)
def test_ligar_cooler(monkeypatch, func, expected):
    monkeypatch.setattr('exemplo_cooler.cpu_temp', func)

    result = ligar_cooler()
    assert expected == result
```

Monkey-patch [ex_04.py]



Dessa forma, não temos que alterar o código e ainda assim conseguimos testar ambas as *branches*. Python é realmente divertido... :)

```python
def ligar_cooler() -> bool:
    temperatura_atual = cpu_temp()

    if temperatura_atual >= TEMPERATURA_LIMITE:
        logger.info(f'Cooler ligado | TEMP {temperatura_atual}')
        return True

    logger.info(f'Cooler desligado |  TEMP {temperatura_atual}')
    return False
```

⚠️ O ponto de atenção é que a resolução do patch é acoplada estruturalmente ao nome do `módulo.função`, caso isso seja alterado, o teste pode não apresentar o resultado esperado.

Recapitulando aqui



Todas as técnicas são igualmente válidas, tendem a funcionar extremamente bem. Mas você ainda quer saber qual é a melhor... Bom... **depende**...

1. Passar o valor exigirá que a DOC só suma, mas tenha que ser testado em outro lugar;
2. Usar DI facilita bastante as chamadas, mas imagine isso com mais dependências;
3. Monkey-patch depende exatamente de como o módulo está, se algo mudar, como algum nome, falhará.

Conhecer as técnicas é mais benéfico do que pensar em qual é a melhor... **O design real depende do contexto, do domínio e da equipe**.

Vadiando o problema

Pera! E... Constante? Logging?

E a constante?



Bom, agora você está com as técnicas em mente, vamos refletir um pouco sobre o problema da constante `TEMPERATURA_LIMITE`.

Para o nosso problema de fato ela é só um valor fixo. O que significa que as técnicas que usamos até agora seriam o suficiente. Mas isso não quer dizer que ela por definição não seja um erro de design.

Vendo isso de uma ótica mais realista, a constante pode não ser necessariamente um valor fixo. Pensando em domínio, cada processador teria uma temperatura pertinente para ser refrigerado. O que varia com que ela fosse algo que varia, mas é constante por execução. Talvez uma variável de ambiente?

E a constante? [ex_05.py]

Algo como isso:

```
from os import environ

TEMPERATURA_LIMITE = int(environ.get('TEMPERATURA_LIMITE', '40'))
```

Agora nossos testes deixam de ser determinísticos. Dependendo do valor setado, os testes passam ou não...

```
TEMPERATURA_LIMITE=100 pytest tests_cooler.py -v
======================== test session starts ========================
tests_cooler.py::test_ligar_cooler[Temperatura baixa] PASSED     [ 50%]
tests_cooler.py::test_ligar_cooler[Temperatura alta] FAILED      [100%]

TEMPERATURA_LIMITE=40 pytest tests_cooler.py -v
======================== test session starts ========================
tests_cooler.py::test_ligar_cooler[Temperatura baixa] PASSED     [ 50%]
tests_cooler.py::test_ligar_cooler[Temperatura alta] PASSED      [100%]
```

Misturando técnicas [ex_05.py]



Conforme você olha o problema por óticas diferentes, podemos chegar a soluções bastante contundentes.

Se fossemos usar a técnica de **passagem de valores** para constante, teríamos algo como:

```python
def ligar_cooler(
    cpu_temp_fn: Callable[[], float] = cpu_temp,  # DI
    max_temp: int = TEMPERATURA_LIMITE,           # Passagem de valores
) -> bool:
    temperatura_atual = cpu_temp_fn()
    if temperatura_atual >= max_temp:
        logger.info(f'Cooler ligado | TEMP {temperatura_atual}')
        return True

    logger.info(f'Cooler desligado |  TEMP {temperatura_atual}')
    return False
```

Misturando técnicas



Tendo testes parecidos com esse:

```
@pytest.mark.parametrize(
    ('func', 'limit', 'expected'),
    [
        (lambda: 30, 40, False),
        (lambda: 30, 20, False),
        (lambda: 50, 40, True),
    ],
)
def test_ligar_cooler(func, limit, expected):
    result = ligar_cooler(func, limit)
    assert expected == result
```

Misturando técnicas



E é aqui que mora o pragmatismo da coisa, as técnicas não são excludentes. O grande ponto, pelo menos para mim, é que embora adicionar mais parâmetros pode tender a deixar a função mais "pura", depender que todos os parâmetros sejam passados, pode gerar efeitos indesejados na nossa API.

```python
def ligar_cooler(cpu_temp_fn: Callable[[], float] = cpu_temp) -> bool:
    temperatura_atual = cpu_temp_fn()

    if temperatura_atual >= TEMPERATURA_LIMITE:
        logger.info(f'Cooler ligado | TEMP {temperatura_atual}')
        return True

    logger.info(f'Cooler desligado |  TEMP {temperatura_atual}')
    return False
```

Que tal juntar DI com patch?

DI + Patch de env [ex_06.py]



Ficaria +- assim:

```python
@pytest.mark.parametrize(
    ('func', 'limit', 'expected'),
    [
        (lambda: 30, '40', False),
        (lambda: 30, '20', False),
        (lambda: 50, '40', True),
    ],
)
def test_ligar_cooler(monkeypatch, func, limit, expected):
    # Aqui alteramos a veriável de ambiente
    monkeypatch.setenv('TEMPERATURA_LIMITE', limit)

    result = ligar_cooler(func)
    assert expected == result
```

API pública: `def ligar_cooler(cpu_temp_fn = cpu_temp)`

Mas por quê?



Porque nem toda dependência tem o mesmo peso conceitual.

- `cpu_temp` é uma **colaboração comportamental**: Ela executa algo. Ela decide algo. Ela é dinâmica.
- `TEMPERATURA_LIMITE` é **configuração**: Ela representa um valor de ambiente. Ela é estática durante a execução.

Misturar ambas como se fossem iguais é o que geralmente confunde decisões de design.

Quando us DI para o invocável (`cpu_temp_fn`), eu estou tornando explícita uma colaboração.

Estou dizendo:

> Essa função depende de alguém que saiba ler a temperatura.

Comportamento / Contrato



Isso é parte do comportamento.

Mas quando faço patch da constante, eu estou lidando com **estado de ambiente**, não com colaboração.

Transformar toda configuração em parâmetro pode deixar a função mais pura, ao mesmo tempo que também pode transformar a assinatura em algo que não representa mais o domínio.

Nem toda dependência merece virar contrato público na API.

O que cada coisa exatamente é?



O ponto importante aqui não é decorar a tabela:

Exemplo	Problema típico	Para ler depois
`cpu_temp()`	Não determinismo	Collaborator / DOC
`TEMPERATURA_LIMITE`	Estado global	Global State
Sensor real, `psutil`	Dependência externa	External Resource
`logger.info()`	Observabilidade	Side Effect
`'exemplo_cooler.cpu_temp'`	Fragilidade de patch	Static Coupling

É perceber que:

> Técnicas diferentes resolvem categorias diferentes de dependência. Às vezes de forma mais simples, às vezes de forma mais complexa.

Quando a gente trata tudo como "o mesmo tipo de dependência", começa a tomar decisões exageradas. Que refletem diretamente no design do código.

Mas e os logs?



Até agora deixamos o logger ali, meio inocente. Mas e se quisermos testar que o log foi emitido?

Não precisamos transformar tudo em dependência explícita, muito menos criar um container maluco.

O próprio pytest já nos dá ferramentas para lidar com esse tipo de efeito colateral.

```
def test_log_quando_liga(caplog, monkeypatch):
    caplog.set_level(DEBUG)
    monkeypatch.setenv('TEMPERATURA_LIMITE', '5')

    ligar_cooler(lambda: 10)

    assert 'Cooler desligado' in caplog.text
```

Pensando pensamentos...



Aqui não precisei:

- Tornar o logger parâmetro
- Criar interface
- Tornar tudo puro

Usamos a ferramenta certa para o tipo certo de dependência.

```python
def test_log_quando_liga(monkeypatch, caplog):
    monkeypatch.setenv('TEMPERATURA_LIMITE', 5)

    ligar_cooler(lambda: 10)

    assert "Cooler ligado" in caplog.text
```

Pureza, é sobre isso... [ex_08.py]



Vamos partir de um experimento levado ao absurdo. Vamos imaginar que removemos todas as dependências externas em uma função pura. Ela não sofre / nem gera nenhum side effect.

```python
def ligar_cooler(  # Passagem de valores
    cpu_temp: float,
    max_temp: int = TEMPERATURA_LIMITE,
    logger: Logger = logger,
) -> bool:
    if cpu_temp >= max_temp:
        logger.info("Cooler ligado")
        return True

    logger.info("Cooler desligado")
    return False
```

Determinística, pura, sem efeitos colaterais, trivial de testar

Pureza ao extremo



É tudo que o SUT precisa, não? Olha esse teste:

```
class Logger:
    def __init__(self):
        self.messages = []

    def info(self, message):
        self.messages.append(message)

@pytest.mark.parametrize(
    ('temp', 'max_temp', 'msg', 'expected'),
    [(30, 40, 'Cooler desligado', False), (50, 40, 'Cooler ligado', True)]
)
def test_decidir_estado_cooler(temp, max_temp, msg, expected):
    logger = Logger()

    assert ligar_cooler(temp, max_temp, logger) == expected
    assert msg in logger.messages
```

Pensando pensamentos



Perfeito.

Mas as perguntas chegam...:

- Quem cria tudo isso?
- Quem passa essas dependências?
- A API pública precisa mesmo disso tudo?

Se o sistema começa a crescer, a composição começa a ficar espalhada.

É aqui que entra o conceito original de Inversão de Controle.

Containers e Inversão de Controle (IoC)



Inversão de Controle significa basicamente:

Quem usa uma dependência não é quem a cria.

Nada místico, nada mágico, nada de framework. Só isso. Podemos modelar a forma mais simples possível de um "container":

```
class Container:
    def __init__(self):
        self._services = {}

    def register(self, name, dependency):
        self._services[name] = dependency

    def __getattr__(self, name):
        return self._services[name]
```

IoC [ex_09.py]



E usar assim:

```
container = Container()
container.register('cpu_reader', cpu_temp)
container.register('max_temp', 40)
container.register('logger', logger)

def ligar_cooler(container: Container) -> bool:
    if container.cpu_reader() >= container.max_temp:
        container.logger.info(
            f'Cooler ligado | TEMP {temperatura_atual}'
        )
        return True

    container.logger.info(
        f'Cooler desligado | TEMP {temperatura_atual}'
    )
    return False
```

IoC



Agora:

- A função não cria nada.
- Ela apenas resolve dependências.
- A composição está centralizada.

Mas... isso melhorou o design? Ou só mudou o lugar da complexidade?

Introduzimos:

- indireção.
- um ponto global implícito.
- outra dependência: o próprio container.

E para um exemplo simples como esse... talvez seja exagero. Me pergunto se existe realmente um caso, dado que é fácil contornar as coisas em python que isso realmente faça sentido...

Conclusão



Testabilidade é uma arte estranha que impacta diretamente o design do código. E o design, por sua vez também influencia como o código vai ser testado.

Como falamos no começo, Python é uma linguagem "estranha" do ponto de vista teórico. Muitas vezes, o que os livros de engenharia sugerem não se aplica tão bem aqui, por conta das características da linguagem, como a tipagem dinâmica e a flexibilidade de misturar paradigmas.

No fim das contas, isso torna o estudo da testabilidade e do design bem mais divertido, porque temos mais liberdade para experimentar e tomar decisões mais práticas, sem ficar preso a uma única teoria ou técnica.



apoia.se/livedepython



pix.dunossauro@gmail.com



patreon.com/dunossauro



Ajude o projeto



Albano Maywitz, Alexandre Costa, Alexandre Harano, Alexandre Lima, Alexandre Takahashi, Alexandre Villares, Alfredo Neto, Alynnefs, Alysson Oliveira, Andre Makoski, Andre Mesquita, Andre Paula, Andre Souza, Antonio Filho, Apolo Ferreira, Aurelio Costa, Belisa Arnhold, Bennie Lima, Bernarducs, Bruno Bereoff, Bruno Freitas, Bruno Lima, Bruno Ramos, Bruno Russian, Brunu, Canibasami, Carlos Gonçalves, Carlos Henrique, Carol Souza, Cauã Oliveira, Celio Araujo, Christian Fischer, Claudemir Cruz, Cleiton Fonseca, Curtos Treino, Daniel Aguiar, Daniel Brito, Daniel Bruno, Daniel Souza, Daniel Wojcickoski, Danilo Linux, Danilo Silva, Darcioalberico, David Couto, Dh44s, Diego Guimarães, Diego Suhett, Dilan Nery, Dunossauro, Edgar, Elias Moura, Emerson Rafael, Érico Andrei, Everton Silva, Ewertonbello, Fabiano Gomes, Fábio Belotto, Fabiokleis, Felipe Adeildo, Felipe Augusto, Felipe Corrêa, Fernanda Prado, Ferrabras, Fightorfall, Francisco Aclima, Franklin Sousa, Franko, Frederico Damian, Gabriel Lira, Gabriel Mizuno, Gabriel Paiva, Gabriel Simonetto, Geilton Cruz, Geisler Dias, Gnomo Nimp, Grinaode, Guilherme Felitti, Guintter, Gustavo Suto, Harold Gautschi, Heitor Fernandes, Hellyson Ferreira, Helton, Helvio Rezende, Henri Alves, Henrique Andrade, Herian Cavalcante, Hiago Couto, Iuri Kintschev, Ivan Santiago, Ivansantiagojr, Janael Pinheiro, Jean Victor, Jefferson Antunes, Jerry Ubiratan, Jhonata Medeiros, Jonas Araujo, Jonatas Leon, Jose Andrade, Jose Barroso, Joseíto Júnior, José Predo), Jota_lugs, Jplay, Jrborba, Ju14x, Juan Felipe, Juliana Machado, Julio Cesar, Julio Franco, Julio Gazeta, Julio Silva, Kaio Engineer, Kaio Peixoto, Leandro Pina, Leandro Vieira, Leonardo Mello, Leonardo Nazareth, Lisandro Pires, Looshigooshi, Lucas Carderelli, Lucas Castro, Lucas Polo, Lucas Schneider, Luciano Ratamero, Lúcia Silva, Luidduarte, Luis Caetano, Luis Eduardo, Luis Ottoni, Luiz Duarte, Luiz Martins, Luiz Paula, Marcel Liguili, Marcelo Fonseca, Marcio Freitas, Marcos Almeida, Marina Passos, Mateusamorim96, Matheus Vian, Medalutadorespacialx, Mlevi Lsantos, Murilo Carvalho, Nhambu, Oopaze, Otávio Carneiro, Patrick Felipe, Prettyhuman, Programming, Rafael Ferreira, Rafael Fontenelle, Rafael Lopes, Rafael Romão, Ramayana Menezes, Ramon Lobo, Renan, Renan Sebastião, Ricardo Silva, Ricardo Viana, Richard Costa, Richard Sousa, Rinaldo Magalhaes, Robsonpiere, Rodrigo Barretos, Rodrigo Santana, Rodrigo Vieira, Rogério Nogueira, Rui Jr, Samael Picoli, Santhiago Cristiano, Scrimf00x, Scrimfx, Shirakawa, Studies, Téo Calvo, Teomewhy, Thamires Betin, Tharles Andrade, Thiago Araujo, Thiago Lucca, Thiago Paiva, Tiago, Tomás Tamantini, Valdir, Varlei Menconi, Vinícius Areias, Vinicius Silva, Vinicius Souza, Vinicius Stein, Vitoria Trindade, Vladimir Lemos, Williamslews, Willian Lopes, Xxxxxxxxx, Zero! Studio



Obrigado você <3



Referências

- BECK, K. Aim, fire [test-first coding]. IEEE Software, v. 18, n. 5, p. 87–89, set. 2001.
- BOURQUE, Pierre; FAIRLEY, R. E. (ORGS.). SWEBOK: guide to the software engineering body of knowledge. Version 3.0 ed. Los Alamitos, CA: IEEE Computer Society, 2014.
- Dependency injection and inversion of control in Python — Dependency Injector 4.48.3 documentation. Disponível em: https://python-dependency-injector.ets-labs.org/introduction/di_in_python.html. Acesso em: 23 fev. 2026.
- MESZAROS, Gerard. XUnit test patterns: refactoring test code. 3. printing ed. Upper Saddle River, NJ Munich: Addison-Wesley, 2009.
- pytest documentation. Disponível em: https://docs.pytest.org/en/stable/. Acesso em: 23 fev. 2026.
- SOMMERVILLE, Ian. Software engineering. Tenth edition ed. Boston Columbus Indianapolis New York San Francisco Hoboken Amsterdam Cape Town Dubai London: Pearson, 2016.
- Why? Disponível em: https://svcs.hynek.me/en/stable/why.html. Acesso em: 23 fev. 2026.